SHA-256

Marcin Kocot

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

Algorytm

Klasa `SHA256` zaczyna się od zdefiniowania stałych, potrzebnych do algorytmu

```
        private val H = listOf<Long>(
            0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
            0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19,
        )
```

```
        private val K = listOf<Long>(
            0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
            0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
            0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
            0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
            0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
            0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
            0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
            0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
            0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
            0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
            0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
            0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
            0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
            0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
            0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
            0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
        )
```

Rozszerzanie

Zdefiniowano klasę `Words16`, przedstawiającą 16 słów (w znaczeniu algorytmu). Jedyną ich funkcją było rozszerzanie się do 64 słów:

```
        class Words16(words: List<BitByteList>) : Words(16, words) {
            fun expand(): Words64 {
                val w = this.words.toMutableList()
                for (i in 16..63) {
                    val s0 =
                        w[i-15].rotateRight(7) xor
                          w[i-15].rotateRight(18) xor
                          w[i-15].shiftRight(3)
                    val s1 =
                        w[i-2].rotateRight(17) xor
                          w[i-2].rotateRight(19) xor
                          w[i-2].shiftRight(10)
                    w.add(BitByteList.numberSum(w[i-16], s0, w[i-7], s1)
                      .takeLastExactly(4))
                }
                return Words64(w.toList())
            }
        }
```

Krok

Dzięki zaimplementowanej wcześniej (i używanej do wszystkich zadań) klasy reprezentującej ciąg bitów (oraz dzięki możliwościom języka Kotlin), krok algorytmu SHA-2 można było zdefiniować używając ładnie wyglądających funkcji typu `infix` (czyli zamiast napisać `A.xor(B)` można było użyć `A xor B`).

```
        private fun step(i: Int, w: Words64): Hash {
            val s1 = e.rotateRight(6) xor e.rotateRight(11) xor
e.rotateRight(25)
            val ch = (e and f) xor (e.not() and g)
            val t1 = BitByteList.numberSum(h, s1, ch, K[i], w[i])
            val s0 = a.rotateRight(2) xor a.rotateRight(13) xor
a.rotateRight(22)
            val maj = (a and b) xor (a and c) xor (b and c)
            val t2 = s0 numberAdd maj

            return Hash(
                t1 numberAdd t2,
                a,
                b,
                c,
                d numberAdd t1,
                e,
                f,
                g,
            )
        }
```

Uruchamianie

Testy wykonano na dwóch tekstach - różniły się tylko jednym znakiem interpunkcyjnym. Wszystkie hasze zweryfikowano w internetowym generatorze SHA-256.

```
1. This project is a considerably improved version of the NIST Statistical
Test Suite (STS), a collection of tests used in the evaluation of the
randomness of bitstreams of data.
2. This project is a considerably improved version of the NIST Statistical
Test Suite (STS). a collection of tests used in the evaluation of the
randomness of bitstreams of data.
3. ee2443c680a0ff4f84a68addbf168904d42d1d43b144ad397802ab779b49405c
4. ad75ee0425fb214ece9888434ff8db48aaf6444c9d1128d5ef464141ceb89e28
5. 50.78125%
```

1. Tekst 1
2. Tekst 2
3. SHA-256 tesktu 1
4. SHA-256 tekstu 2
5. Ile procent bitów było różnych w haszach (zarówno wartość dużo poniżej jak i dużo powyżej 50% mogłyby oznaczać zależenie tych haszy od siebie)